Exclusion Zones for Floating Overlays: Per-Element, Scroll-Linked Avoidance on Large Screens

Moataz Mohamed
House of Algorithms

Idea paper · November 2026 · Reference implementation: github.com/moatazhamada/FloatAround

Abstract

Floating navigation surfaces such as pill-shaped rails are increasingly common on tablets and unfolded foldables. They free space compared with edge-docked navigation, but they also cover whatever scrolls underneath them. The two prevailing responses, a page-wide inset or letting content pass underneath, either waste the space the overlay was meant to free or hide content. We describe *exclusion zones*, a layout pattern in which every element computes its own inset from its current distance to the overlay, easing in over a short feather distance with a smootherstep curve. The inset is a pure function of geometry rather than an animation, so it stays synchronised with scrolling, folding and resizing. We distinguish four element roles (shifting, compressing, padding-shifting and full-bleed), present the algorithm, discuss related techniques in text layout and platform inset systems, and provide an open-source Jetpack Compose implementation. We close with the limitations of the approach and a proposed user study.

Keywords: adaptive layout, large screens, foldables, floating navigation, exclusion paths, UI design patterns, Jetpack Compose.

1. Introduction

Large-screen guidance for Android recommends moving navigation from a bottom bar to a side rail once the window is wide enough [6]. Recent design languages go further and detach the rail from the edge: it floats as a rounded pill with a shadow, and the page extends behind it [3]. The visual result is lighter, but the overlay now intersects scrolling content. Titles, the first item of a horizontal carousel and the first words of every line in a paragraph can all end up underneath it.

The pill is typically short relative to the window. On an unfolded foldable in landscape, a five-item pill occupies roughly a third of the height. A page-wide start inset therefore reserves a strip of space for the whole page to solve a problem that only affects the rows currently beside the pill.

This paper describes a pattern that resolves the conflict at the level of individual elements. It is presented in the problem / forces / solution / consequences structure common in pattern literature, followed by the algorithm, related work and limitations.

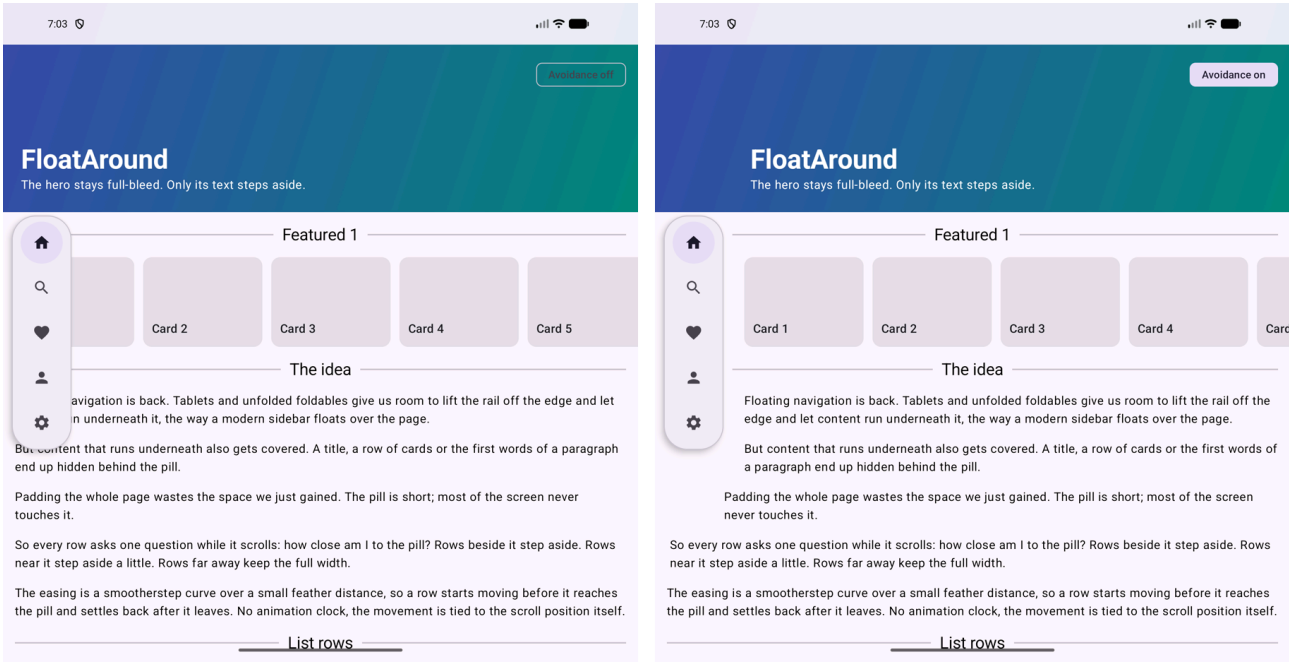


Figure 1. Left: avoidance off, content runs under the floating pill. Right: avoidance on, paragraphs, list rows and the leading header line step aside only near the pill.

2. The pattern

Context

A scrolling page on a window wide enough for side navigation, with a floating overlay (rail, pill, toolbar, mini-player) anchored to one horizontal edge and covering only part of the window height.

Problem

How can content remain fully visible beside the overlay without giving up the space the overlay leaves free everywhere else?

Forces

- **Visibility:** interactive and textual content must not be covered.
- **Space:** rows above and below the overlay should keep the full window width.
- **Continuity:** movement must feel continuous during scrolling, folding, rotation and window resizing; abrupt jumps are distracting.
- **Element semantics:** a paragraph, a section header, a carousel and a hero image should not react the same way.
- **Cost:** the solution must not trigger work for the whole page on every scroll frame.

Solution

Treat the overlay as an obstacle that each element avoids individually. Every participating element reads its own bounds and the obstacle's bounds and computes a start inset: the full horizontal intrusion plus a gap when it overlaps the obstacle vertically, a smoothly eased fraction of it within a feather distance, and zero beyond. The inset is applied as layout, not as an animation, so it is always a function of the current geometry.

Element roles

Role	Behaviour	Examples
Shifting	Gives up its start edge; its end edge stays fixed, so content narrows and may rewrap.	Paragraphs (each paragraph separately), list rows, cards, headings
Compressing	Only a decorative sub-element absorbs the inset; the semantic content does not move.	The leading rule of a centred section header
Padding-shifting	The container keeps its full frame; the inset is added to its content padding.	Horizontal carousels: the first item steps aside, later items still scroll to the edge
Full-bleed	Does not participate and runs under the overlay.	Hero images and video; text placed on them is shifting

Consequences

- Content is never covered while rows far from the overlay keep the full width.
- Because the inset is derived from position, scrolling, folding, unfolding and resizing need no special handling: when the obstacle or the element moves, the inset follows.
- Shifting text can rewrap and change height, which moves the elements below it (Section 5).
- Each element must declare its role. This is a design decision per component, not a global switch.

3. Algorithm

Let the element bounds be *r* and the obstacle bounds *o*, both in a shared coordinate space, with gap *g* and feather *f*. For an obstacle on the start edge in a left-to-right layout:

```
intrusion = o.right + g − r.left              (RTL: r.right − o.left + g)
if intrusion <= 0        return 0            // already clear horizontally
d = max(o.top − r.bottom, r.top − o.bottom, 0)  // vertical gap, 0 when overlapping
if d == 0                return intrusion
if d >= f                return 0
t = 1 − d / f
return intrusion ∗ t³ (6t² − 15t + 10)       // smootherstep [5]
```

Smootherstep has zero first and second derivatives at both ends, so an element starts and stops moving without a visible kink in velocity or acceleration. With $f = 0$ the function degrades to a hard step: overlapping elements step aside, all others do not. In the reference implementation g = 12dp and f = 140dp.

The function has no state and no time parameter. This is the key difference from an animated transition: there is nothing to start, cancel or interrupt, and the result for a given scroll position is always the same.

4. Reference implementation

FloatAround implements the pattern for Jetpack Compose. The overlay publishes its bounds with `Modifier.floatingObstacle`. Shifting and compressing elements use `Modifier.avoid`, a layout modifier node that observes its own global position and reads the inset during the measure pass; it measures its child with reduced width and places it at the inset. Since the inset is read in the layout phase, a change re-lays out only the affected elements and triggers no recomposition. Padding-shifting containers use a tracker that

observes the container position and a `PaddingValues` implementation that adds the inset on the start side. The pure function is exposed separately for ports to other toolkits.

```
val obstacle = remember { FloatingObstacle(gap = 12.dp, feather = 140.dp) }
Pill(Modifier.align(Alignment.CenterStart).floatingObstacle(obstacle))
Text(paragraph, Modifier.avoid(obstacle))
HorizontalDivider(Modifier.weight(1f).avoid(obstacle))
LazyRow(Modifier.track(tracker), contentPadding = base.plusAvoidance(tracker, density))
```

5. Related work

Text exclusion paths. Apple's TextKit lets a text container exclude arbitrary paths so that lines flow around them [1], and CSS Shapes lets inline content wrap around a float's shape [2]. These are the closest conceptual relatives: per-line avoidance of an obstacle. They operate on lines of text inside a single text container and are evaluated once per layout, not per element across a scrolling page.

Background extension. iPadOS 26 floats the sidebar and lets content extend beneath it, mirroring and blurring the content into the covered area so that it appears continuous [3][4]. This preserves full-bleed imagery, which corresponds to the full-bleed role here, but does not move interactive or textual content out from under the overlay.

Window insets and rulers. Android reports system bars, cutouts and the IME as window insets, and Compose's `WindowInsetsRulers` with `Modifier.fitInside` lets a layout fit within such regions, including custom ones [7]. The documentation cautions against using them inside scroll containers; insets are designed for page-level safe areas rather than for elements whose distance to a region changes every frame.

Collapsing and scroll-aware bars. Material's scroll behaviours hide or collapse app bars in response to scrolling [6]. They move the overlay out of the way; exclusion zones move the content instead and keep the navigation visible.

6. Limitations and future work

- **Rewrapping.** A shifting paragraph that narrows can gain a line and push following elements down. The feather makes the change gradual, but it remains a vertical side effect of a horizontal adjustment.
- **One-frame latency.** Element positions are observed after layout, so the inset reflects the previous frame's position. At ordinary scroll speeds this was not perceptible in our tests; a ruler-based implementation could remove it.
- **Single edge.** The reference implementation handles one obstacle on the start edge. Multiple obstacles and top or bottom overlays (floating toolbars, mini-players) are natural extensions: the inset becomes the maximum over obstacles per edge.
- **No user evaluation yet.** We propose a within-subjects study comparing three conditions (content under the overlay, page-wide inset, exclusion zones) on reading speed, mis-taps near the overlay, perceived motion and preference, on an unfolded foldable and a tablet.

7. Conclusion

Floating overlays turn navigation into an obstacle inside the content area. Treating that obstacle the way typography treats a figure, as something each line or element flows around, keeps content visible without giving up the space floating navigation was meant to create. Defining the inset as a pure, eased function of geometry rather than as an animation makes the behaviour stable across scrolling, folding and resizing, and keeps the implementation small.

References

1. Apple. *NSTextContainer.exclusionPaths*. Apple Developer Documentation.
2. W3C. *CSS Shapes Module Level 1*. W3C Candidate Recommendation.
3. Apple. *Elevate the design of your iPad app*. WWDC 2025, session 208.
4. Apple. *backgroundExtensionEffect(isEnabled:)*. SwiftUI documentation.
5. K. Perlin. Improving Noise. *ACM Transactions on Graphics* 21(3), Proc. SIGGRAPH 2002, 681–682.
6. Google. *Adaptive app quality guidelines* and Material 3 navigation rail guidance. Android Developers.
7. Google. *About WindowInsetsRulers*. Android Developers, Jetpack Compose documentation.